



Proyecto #3

Lenguajes de programación

Operación Atlas

Estudiantes:

Tayler Wynta Rodríguez

Maikel Flores Navarro

Luis Trejos Rivera

Profesor:

Ing. Allan Rodriguez Davila

I Semestre 2026

Contenidos

Manual de usuario	3
Instrucciones de compilación.....	3
Requisitos previos	3
Ejecución y uso	4
Pruebas de funcionalidad	4
Descripción del problema	20
Diseño del programa	20
Decisiones de diseño	20
Algoritmos usados	21
Análisis de resultados	22
Objetivos alcanzados.....	22
Objetivos no alcanzados y razones	23
Análisis de predicados	24
Explicación de los tres predicados más complejos	24

Manual de usuario

Instrucciones de compilación

Requisitos previos

- Sistema operativo: Windows 10/11, macOS o Linux
- Conexión a internet para la descarga de dependencias

Instrucciones

1. Instalar Python

- Descargar e instalar Python 3.10 o superior desde: <https://www.python.org/downloads/>
- Durante la instalación en Windows, marcar la casilla *"Add Python to PATH"*.
- Verificar la instalación abriendo una terminal y ejecutando: *python --version*

2. Instalar SWI-Prolog

- Descargar e instalar SWI-Prolog desde: <https://www.swi-prolog.org/download/stable>
- Durante la instalación en Windows, marcar la opción *"Add SWI-Prolog to the system PATH"*.
- Verificar la instalación ejecutando: *swipl --version*

3. Clonar el repositorio

- Abrir una terminal y ejecutar: *git clone <https://github.com/MaikelFn/Operacion-Atlas.git>* o ingresar al enlace y descargar el zip
- Ingresar a la carpeta del proyecto si fue clonado, o descomprimir el zip en caso de descarga y navegar a: *cd Operacion-Atlas*

4. Instalar dependencias de Python

- Ejecutar el siguiente comando en la terminal dentro de la carpeta del proyecto:
pip install pyswip

tkinter ya viene incluido con Python por defecto, no requiere instalación adicional.
En Linux, si no está disponible, ejecutar: *sudo apt install python3-tk*

Ejecución y uso

Una vez completada la instalación, abrir una terminal en la carpeta del proyecto y ejecutar:

- `cd src/Frontend`
- `python main.py`

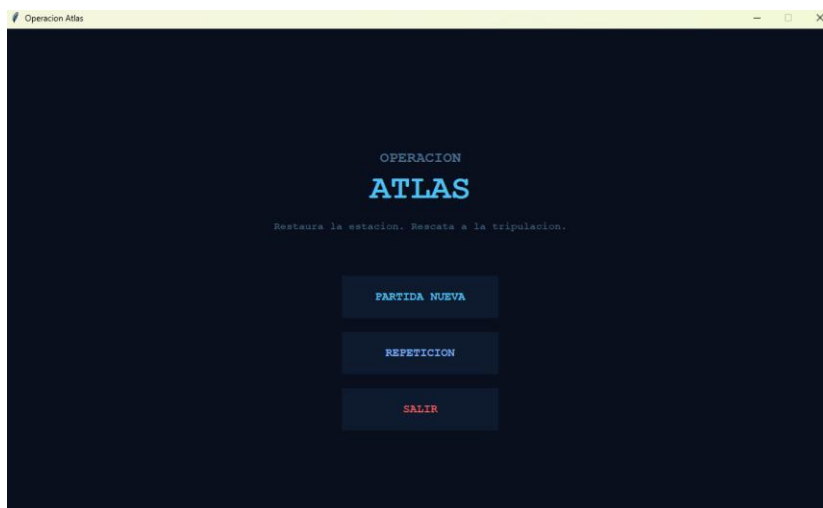
La ventana del juego se abrirá automáticamente (1200x700 px).

Pruebas de funcionalidad

Menú principal:

Al ejecutar el programa, el sistema inicializa automáticamente cargando la base de conocimiento desde Prolog y registrando el módulo inicial como visitado. Como se muestra en la Figura 1, la ventana principal presenta el menú de inicio con las opciones de iniciar una partida nueva, cargar una repetición guardada o salir del programa.

Figura 1:

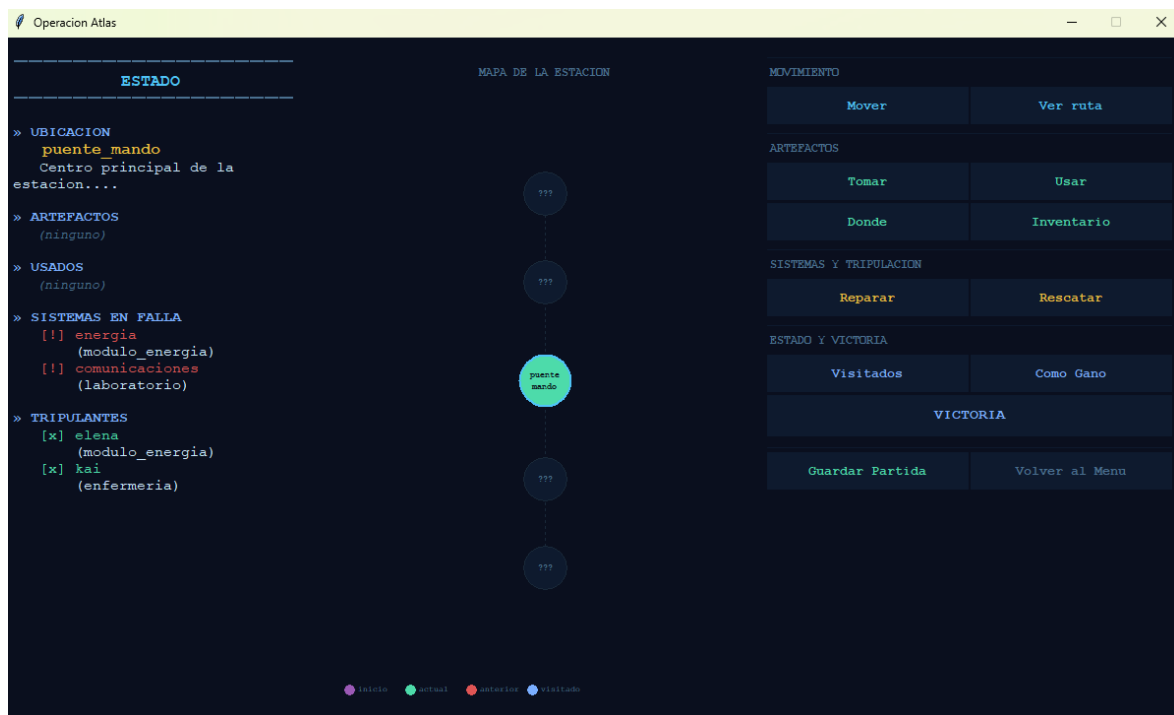


Pantalla de juego:

Una vez seleccionada la opción PARTIDA NUEVA, el sistema carga la pantalla principal del juego y registra automáticamente el módulo inicial como visitado. Como se aprecia en la Figura 2, la interfaz se organiza en tres paneles. El panel izquierdo muestra el estado actual del jugador bajo el título ESTADO, incluyendo la ubicación actual resaltada en amarillo como `puede_mando` con su descripción "Centro principal de la estacion", la lista de artefactos en inventario que al inicio aparece como ninguno, los artefactos usados también vacíos, los sistemas en falla marcados en rojo donde se indican energía en `modulo_energia` y comunicaciones en laboratorio, y los tripulantes atrapados elena en `modulo_energia` y kai en enfermeria.

El panel central presenta el mapa progresivo de la estación bajo el título MAPA DE LA ESTACION. El nodo actual puente_mando aparece resaltado en verde en el centro, mientras que los módulos aún no descubiertos se muestran como nodos oscuros con la etiqueta "???", conectados mediante líneas discontinuas grises. En la parte inferior del mapa se muestra la leyenda que indica el significado de cada color: inicio en morado, actual en verde, anterior en rojo y visitado en azul.

El panel derecho agrupa los botones de acción organizados en cuatro secciones: MOVIMIENTO con los botones Mover y Ver ruta, ARTEFACTOS con Tomar, Usar, Donde e Inventario, SISTEMAS Y TRIPULACION con Reparar y Rescatar, y ESTADO Y VICTORIA con Visitados, Como Gano y Victoria. En la parte inferior se encuentran los botones Guardar Partida y Volver al Menu.



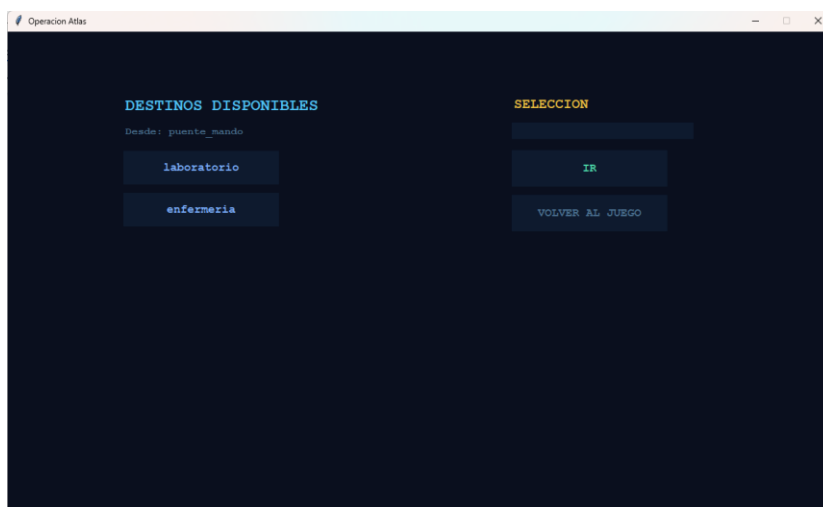
Mover:

Al presionar el botón Mover, el sistema consulta a Prolog los destinos disponibles desde la ubicación actual mediante el predicado puedo_ir/1, validando las restricciones de acceso, pasos previos y estados de sistemas requeridos para cada módulo. Como se muestra en la Figura 3, la pantalla despliega bajo el título DESTINOS DISPONIBLES los módulos a los que el jugador puede trasladarse desde puente_mando, en este caso laboratorio y enfermeria. El panel derecho muestra el campo SELECCION donde se refleja el módulo elegido y los botones IR para confirmar el movimiento y VOLVER AL JUEGO para cancelar.

Al seleccionar laboratorio y presionar IR, el sistema invoca el predicado mover/1 de Prolog que actualiza la posición del jugador en la base de conocimiento dinámica. Como se aprecia en la Figura 4, el sistema despliega un cuadro de confirmación indicando "Te moviste a laboratorio.", confirmando que el movimiento fue ejecutado exitosamente.

Una vez confirmado el movimiento, el sistema regresa automáticamente a la pantalla principal actualizando todos los paneles con el nuevo estado. Como se observa en la Figura 5, el panel izquierdo refleja la nueva ubicación en amarillo como laboratorio con su descripción "Laboratorio científico parcialmente destruido", y el mapa central se recalcula mostrando laboratorio como nodo actual en verde en el centro, puente_mando como nodo anterior en rojo conectado por una línea roja sólida, y un módulo aún no descubierto como nodo oscuro con "???". En la parte inferior del mapa aparece el indicador "+ 1 modulo(s) fuera de vista" señalando que existen módulos del grafo que quedan fuera del rango visible de dos niveles de profundidad.

Figura 3:



Figura

4:

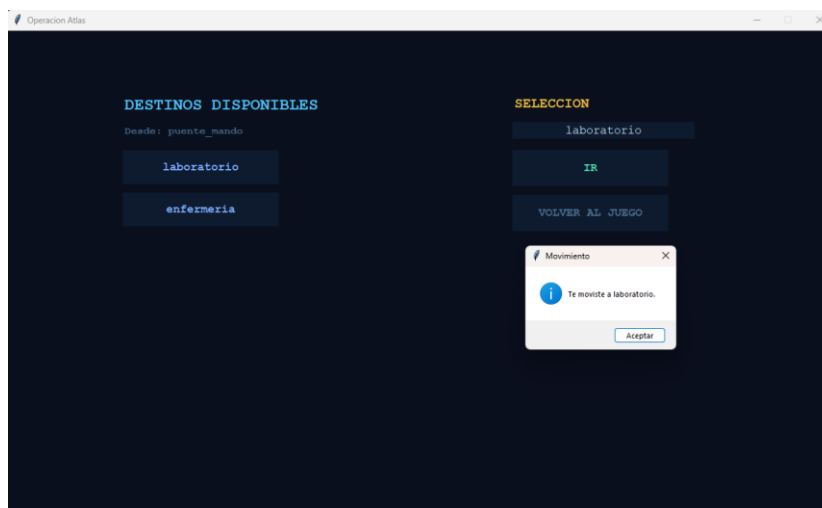
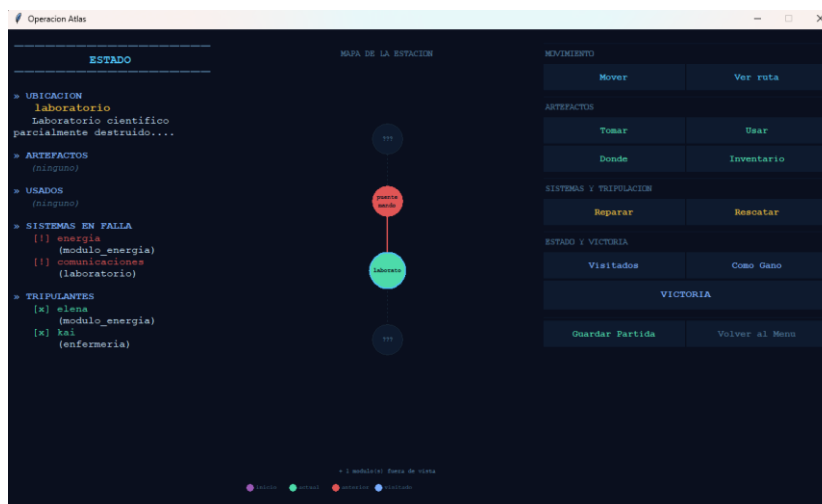


Figura 5:



Ver ruta:

Al presionar el botón Ver ruta, el sistema despliega la pantalla de cálculo de caminos. Como se muestra en la Figura 6, el panel derecho presenta dos listas desplegables donde el jugador selecciona el módulo de origen y el módulo de destino entre todos los módulos disponibles de la estación. Al presionar Mostrar ruta, el sistema invoca el predicado ruta/3 de Prolog y despliega en el panel izquierdo la secuencia de módulos como nodos conectados por flechas. Cada nodo se colorea según el estado de acceso: azul para el origen, verde con "✓ accesible" para módulos a los que el jugador puede ingresar, y rojo con "X bloqueado" para módulos que requieren cumplir condiciones adicionales antes de poder ser visitados.

Figura 6:



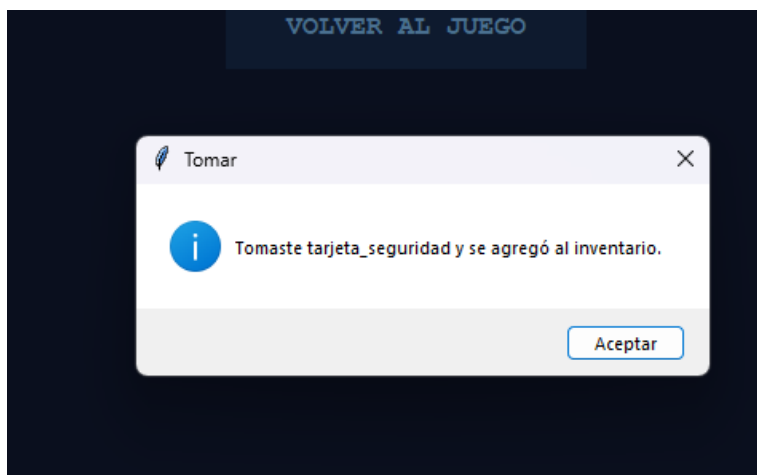
Tomar artefacto:

Al presionar el botón Tomar, el sistema consulta a Prolog los artefactos disponibles en el módulo actual que el jugador aún no posee. Como se muestra en la Figura 7, la pantalla despliega bajo el título ARTEFACTOS DISPONIBLES los artefactos presentes en el módulo actual, en este caso tarjeta_seguridad en puente_mando. Al seleccionarlo, el nombre aparece reflejado en el panel de SELECCION. Al presionar TOMAR, el sistema invoca el predicado tomar/1 de Prolog que agrega el artefacto a la lista dinámica de artefactos logrados. Como se aprecia en la Figura 8, el sistema confirma la acción mediante un cuadro de diálogo con el mensaje "Tomaste tarjeta_seguridad y se agregó al inventario.", tras lo cual el panel de estado se actualiza reflejando el nuevo artefacto en el inventario del jugador.

Figura 7:



Figura 8:



Usar artefacto:

Al presionar el botón Usar, el sistema muestra los artefactos del inventario que aún no han sido activados. Como se observa en la Figura 9, la pantalla despliega bajo el título ARTEFACTOS DEL INVENTARIO los artefactos disponibles para usar, en este caso tarjeta_seguridad obtenida en puente_mando. Al seleccionarlo y presionar USAR, el sistema invoca el predicado usar/1 de Prolog que registra el artefacto como consumido en la lista dinámica de usados, desbloqueando el acceso a los módulos que lo requieren como restricción de entrada. Como se aprecia en la Figura 10, el sistema confirma la acción con el mensaje "Usaste tarjeta_seguridad."

Figura 9:

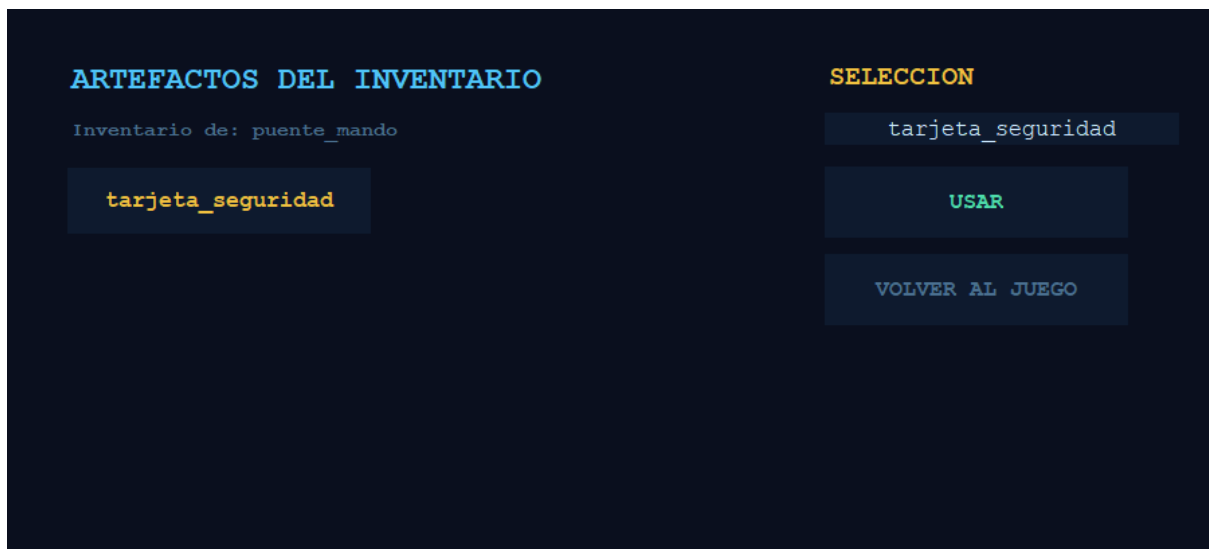
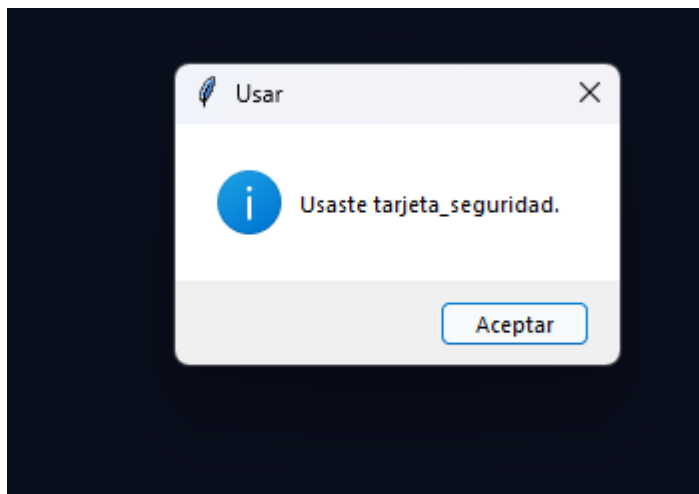


Figura 10:



Donde

está:

Al presionar el botón Donde, el sistema consulta a Prolog todos los artefactos del juego que el jugador aún no posee en su inventario y los despliega en una lista. Como se muestra en la Figura 11, al seleccionar un artefacto de la lista este aparece reflejado en el panel de SELECCION junto con la información de su ubicación actual, mostrando el texto "Se encuentra en:" seguido del nombre del módulo donde puede ser recolectado, consultado mediante el predicado donde_esta/2 de Prolog. Esto permite al jugador planificar su ruta de recolección sin necesidad de explorar todos los módulos manualmente. Un botón VOLVER AL JUEGO permite regresar a la pantalla principal en cualquier momento.

Figura 11:



Inventario:

Al presionar el botón Inventario, el sistema consulta a Prolog el contenido actual del inventario mediante el predicado `que_tengo/1`. Como se muestra en la Figura 12, la pantalla despliega bajo el título INVENTARIO DEL JUGADOR todos los artefactos recolectados durante la partida, indicando junto a cada uno si ya fue usado con la etiqueta [USADO]. Al seleccionar un artefacto, el panel de SELECCION muestra su detalle completo incluyendo si fue usado y el módulo donde fue obtenido, en este caso `tarjeta_seguridad` con Usado: Sí y Obtenido en: `punto_mando`.

Figura 12:



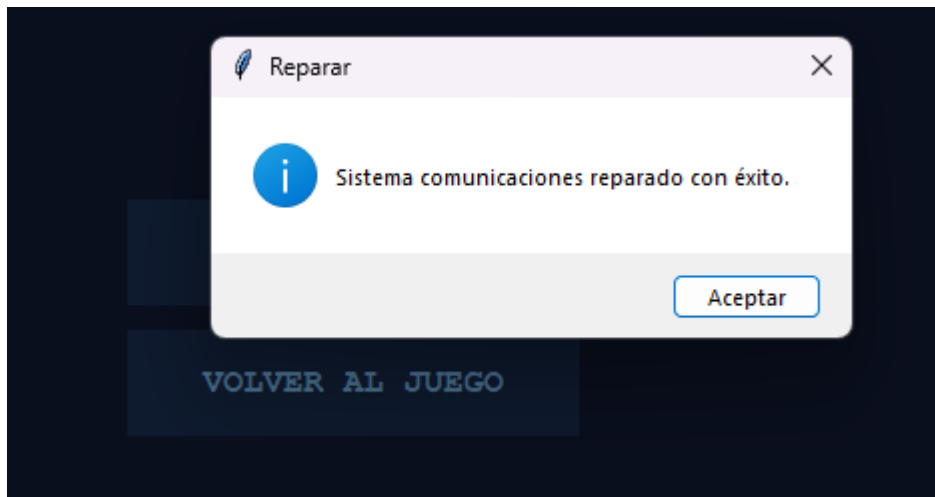
Reparar:

Al presionar el botón Reparar, el sistema consulta a Prolog los sistemas en falla del módulo actual. Como se muestra en la Figura 13, la pantalla despliega bajo el título SISTEMAS EN FALLA (MODULO ACTUAL) los sistemas disponibles para reparar, en este caso comunicaciones. Al seleccionarlo, el panel de SELECCION muestra el nombre del sistema y lista los artefactos requeridos para su reparación, en este caso fusible y traje_espacial. Si el jugador posee todos los artefactos necesarios y los ha usado previamente, al presionar REPARAR el sistema invoca el predicado reparar/1 de Prolog que valida las condiciones y actualiza el estado del sistema en la base de conocimiento dinámica. Como se aprecia en la Figura 14, el sistema confirma la operación con el mensaje "Sistema comunicaciones reparado con éxito.", tras lo cual el panel de estado se actualiza eliminando el sistema de la lista de sistemas en falla.

Figura 13:



Figura 14:



Rescatar:

Al presionar el botón Rescatar, el sistema consulta a Prolog los tripulantes atrapados en el módulo actual. Como se muestra en la Figura 15, la pantalla despliega bajo el título TRIPULANTES ATRAPADOS (MODULO ACTUAL) los tripulantes disponibles para rescatar, en este caso elena. Al seleccionarlo, el panel de SELECCION muestra el nombre del tripulante y lista los sistemas que deben estar restaurados para poder liberarlo, en este caso energía. Si todas las condiciones se cumplen, al presionar RESCATAR el sistema invoca el predicado rescatar/1 de Prolog que valida el estado de los sistemas requeridos y actualiza el estado del tripulante en la base de conocimiento dinámica. Como se aprecia en la Figura 16, el sistema confirma la operación con el mensaje "Tripulante elena rescatado con éxito.", tras lo cual el panel de estado se actualiza eliminando al tripulante de la lista de atrapados.

Figura 15:

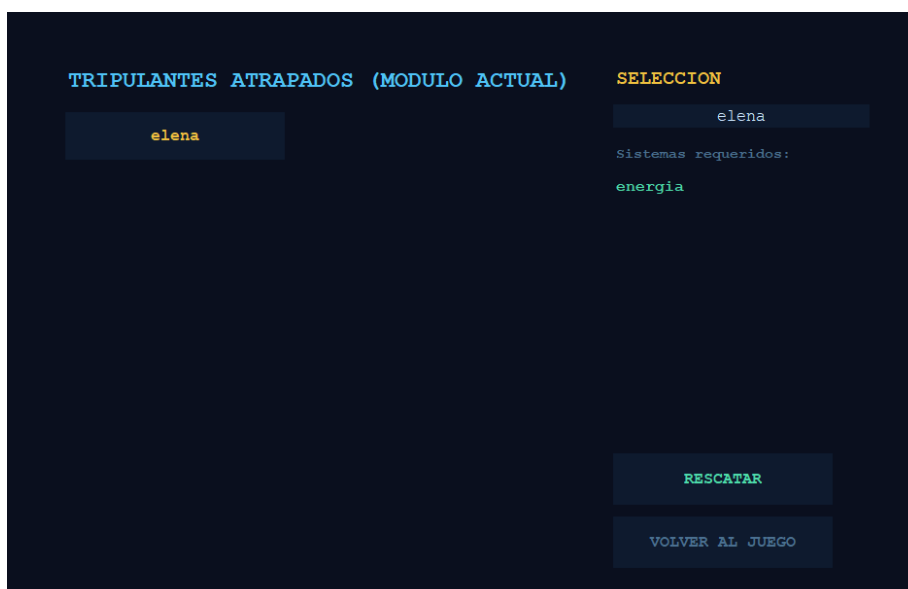
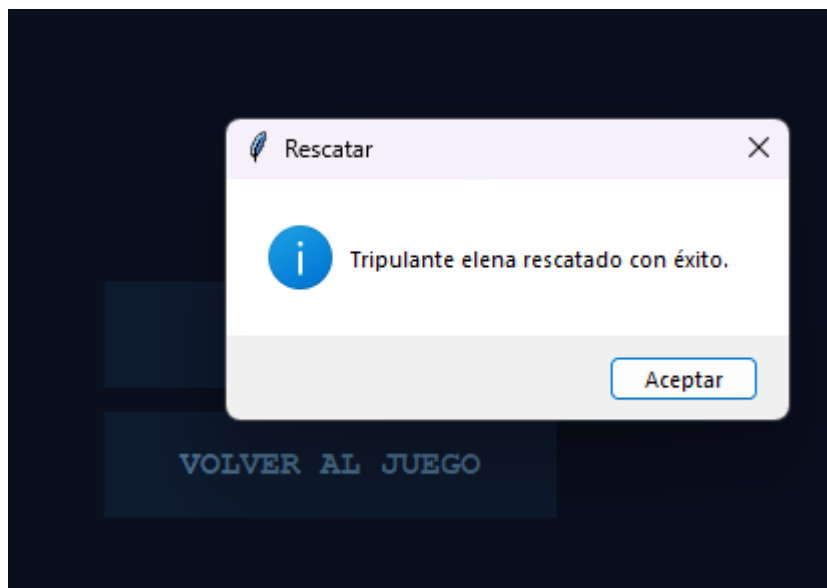


Figura 16:



Módulos visitados:

Al presionar el botón Visitados, el sistema consulta a Prolog el historial de recorrido mediante el predicado `modulos_visitados/1`. Como se muestra en la Figura 17, la pantalla despliega bajo el título **MODULOS VISITADOS** la lista completa de módulos recorridos durante la partida en orden cronológico, en este caso `modulo_energia`, `laboratorio`, `almacen_suministros`, `enfermeria` y `punte_mando`. Al seleccionar cualquier módulo de la lista, el panel derecho muestra su nombre resaltado en amarillo junto con su descripción. Un botón **VOLVER AL JUEGO** permite regresar a la pantalla principal.

Figura 17:



Cómo gano:

Al presionar el botón Como Gano, el sistema invoca el predicado como_gano/1 de Prolog que genera todos los planes posibles para completar la partida. Como se muestra en la Figura 18, la pantalla despliega bajo el título PLAN PARA GANAR las soluciones encontradas, cada una identificada como SOLUCION 1, SOLUCION 2 y así sucesivamente. Cada plan presenta una secuencia numerada de pasos con código de color según el tipo de acción: azul para movimientos (Ir a), verde para recolección de artefactos (Tomar), amarillo para uso de artefactos (Usar), rojo para reparaciones (Reparar) y verde para rescates (Rescatar). La pantalla incluye una barra de desplazamiento para navegar por planes extensos, como se aprecia en la Figura 19 donde se muestra la continuación de la Solución 1 con los pasos de rescate y el mensaje "--- VICTORIA ---" al final del plan. La Figura 20 muestra el inicio de la Solución 2, demostrando que el sistema genera múltiples planes alternativos explorando distintos órdenes de atención. Un botón Actualizar en la esquina superior derecha permite recalcular los planes en cualquier momento según el estado actual de la partida.

Figura 18:

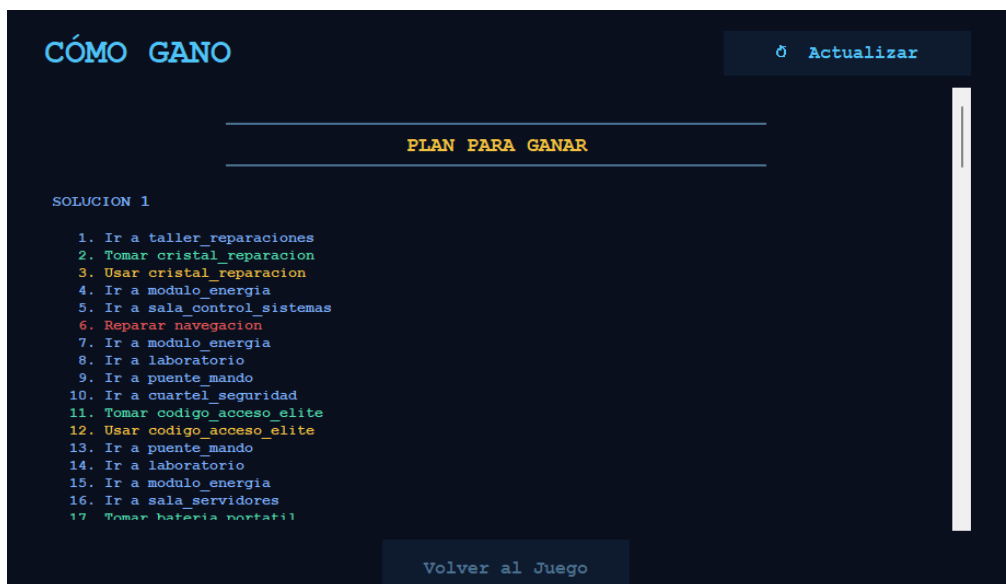


Figura 19:

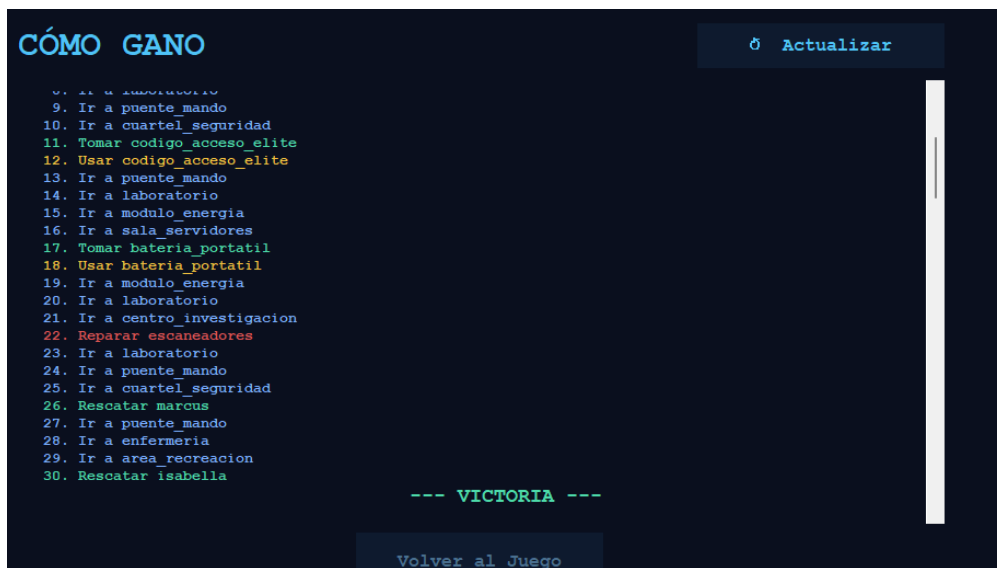


Figura 20:



Victoria:

Al presionar el botón Victoria, el sistema invoca el predicado `verifica_gane/0` de Prolog que evalúa si se han cumplido todas las condiciones declaradas en `objetivoS/2` y `objetivoT/2`. Si aún quedan objetivos pendientes, como se mostró en la Figura 21, la pantalla despliega VICTORIA PENDIENTE listando los sistemas en falla y tripulantes atrapados que aún requieren atención. Una vez completados todos los objetivos, como se aprecia en la Figura 22, la pantalla muestra el mensaje "¡CONDICIÓN DE VICTORIA ALCANZADA!" seguido del RESUMEN FINAL con los artefactos logrados durante la partida — en este caso `bateria_portatil`, `sensor_radiacion`, `codigo_acceso_elite`, `destornillador_plasma`,

cristal_reparacion, traje_espacial, fusible y tarjeta_seguridad — y los 15 módulos visitados listados con marca de verificación.

La Figura 23 muestra la continuación del resumen con los sistemas reparados: escaneadores, datos, defensa, herramientas, navegacion, energia y comunicaciones, y la tripulación rescatada completa: sophia, david, leo, isabella, kai, marcus y elena. Debajo se presenta la ruta realizada con el historial cronológico de cada acción ejecutada durante la partida, incluyendo movimientos, tomas, usos y reparaciones. Al final del resumen aparece el mensaje "¡Has completado la misión con éxito!" confirmando la victoria.

Figura 21:

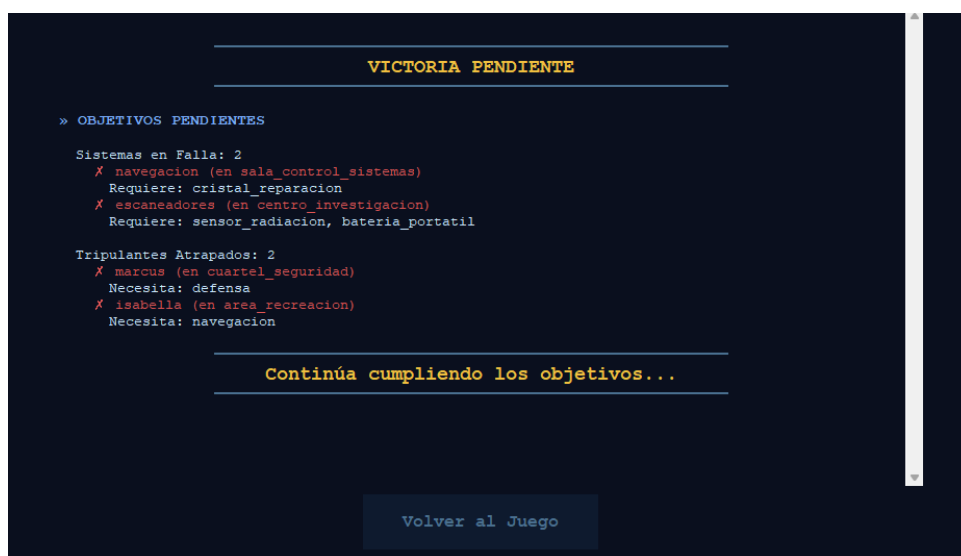


Figura 22:



Figura 23:



Guardar y repetición:

Al presionar el botón Guardar Partida desde la pantalla principal del juego, el sistema muestra un cuadro de confirmación antes de proceder. Como se muestra en la Figura 24, el diálogo advierte "Esto reescribirá tu partida anterior ¿Estás seguro de que deseas continuar?" con opciones Sí y No, evitando sobreescrituras accidentales. Al confirmar, el sistema invoca el predicado `guardar_repeticion/0` de Prolog que escribe en disco la secuencia completa de comandos ejecutados durante la partida. Como se aprecia en la Figura 25, el sistema confirma la operación con el mensaje "Tu partida ha sido guardada exitosamente."

Para reproducir la partida guardada, el jugador selecciona el botón REPETICION desde el menú principal. El sistema carga el archivo desde disco e invoca el predicado `reproducir_repeticion/0` de Prolog que re-ejecuta todos los comandos guardados, restaurando el estado completo de la partida. Como se observa en la Figura 26, al finalizar la reproducción el sistema muestra el mensaje "Partida reproducida exitosamente." y el panel de estado refleja todos los artefactos recolectados y usados, y el mapa actualizado con los módulos visitados correspondientes a la sesión guardada.

Figura 24:

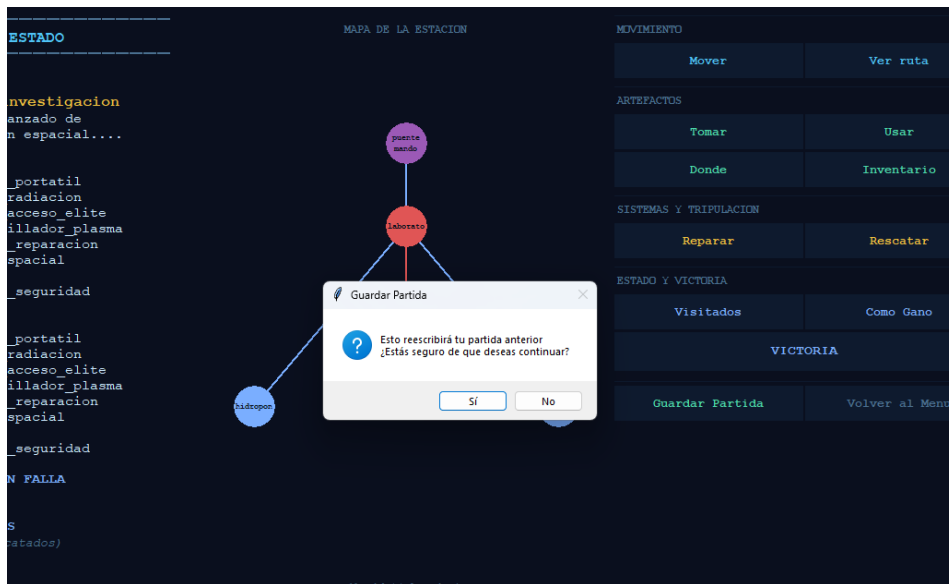


Figura 25:

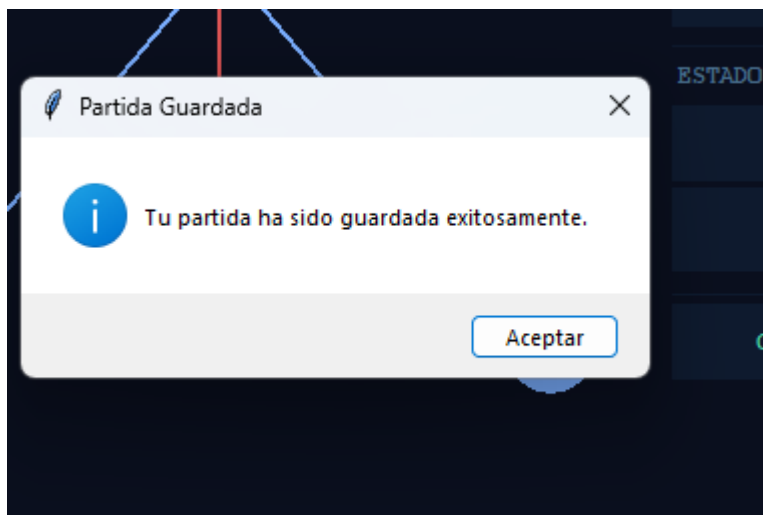
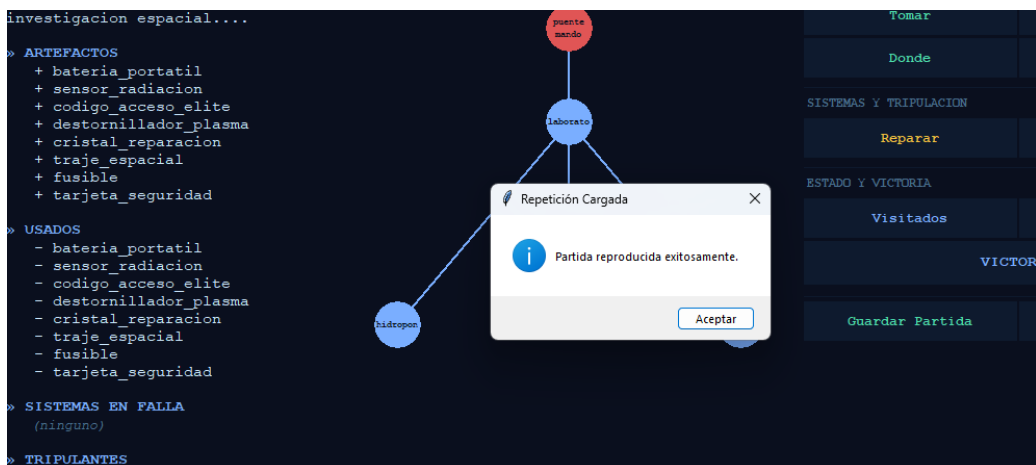


Figura 26:



Descripción del problema

La humanidad ha establecido una base orbital denominada Estación Aurora, destinada al desarrollo de experimentos científicos en el espacio. Tras una tormenta solar de gran magnitud, varios módulos de la estación quedaron aislados entre sí y el sistema central sufrió daños críticos. Como consecuencia, sistemas esenciales dejaron de funcionar y miembros de la tripulación quedaron atrapados en distintos sectores de la estación.

El jugador asume el rol del ingeniero principal a bordo, cuya misión es recorrer los módulos de la estación, recolectar artefactos dispersos, reparar los sistemas dañados y rescatar a la tripulación atrapada antes de que la situación se vuelva irreversible.

El problema central consiste en modelar este entorno dinámico utilizando el paradigma lógico de programación, específicamente con Prolog como lenguaje principal. Esto implica representar el conocimiento del mundo del juego mediante hechos y reglas, y resolver cada acción del jugador a través de inferencias lógicas.

Diseño del programa

Decisiones de diseño

1. **Separación estricta de capas:** El sistema se divide en dos capas completamente independientes: el motor lógico en Prolog (`logica.pl` y `conocimiento.pl`) que contiene toda la inteligencia del juego, y la interfaz gráfica en Python con `tkinter` que únicamente recibe la entrada del usuario y muestra las respuestas. Ninguna regla del juego se procesa del lado de Python.
2. **Comunicación mediante `pyswip`:** Se utilizó la librería `pyswip` para conectar Python con SWI-Prolog en tiempo de ejecución. Desde Python se lanzan consultas al motor Prolog mediante los métodos `consultar_uno()` y `consultar_todos()`, que envuelven las llamadas a `query()` de `pyswip`. Esto permite que la interfaz desconozca completamente la lógica del juego.
3. **Separación de la base de conocimiento:** La base de conocimiento estática (módulos, enlaces, artefactos, sistemas, tripulantes y restricciones) se separó en un archivo propio `conocimiento.pl`, mientras que `logica.pl` contiene exclusivamente los predicados

dinámicos y las reglas del juego. Esto facilita modificar el mundo del juego sin tocar la lógica.

4. **Estado dinámico con retract/assert:** Dado que Prolog es declarativo, el estado mutable del jugador se maneja modificando la base de conocimiento en tiempo de ejecución. Los predicados dinámicos jugador/1, artefactosLogrados/1, visitados/1, usados/1, sistema/4 y tripulante/4 se actualizan con retract y assertz ante cada acción del jugador, garantizando persistencia durante la partida.
5. **Historial cronológico de acciones:** Se implementó el predicado dinámico ruta_historial/1 que registra en orden cronológico cada acción relevante del jugador (moverse, tomar, usar, reparar, rescatar) mediante agregar_a_ruta/1. Esto permite mostrar al final la trayectoria completa de la partida sin necesidad de reconstruirla.
6. **Listas como estructura de datos principal:** Todas las colecciones del juego (inventario, módulos visitados, artefactos usados, sistemas reparados, tripulantes rescatados) se representan como listas Prolog. Se evita el uso de estructuras más complejas para mantener compatibilidad con el paradigma lógico puro y simplificar las consultas con member/2.
7. **Corte (!) para evitar backtracking innecesario:** En predicados donde solo se espera un resultado o se quiere priorizar una rama (como registrar_visita/1 y verifica_gane/0), se utilizó el corte (!) para detener la búsqueda una vez encontrada la primera solución válida, mejorando el rendimiento.
8. **Navegación por frames en la interfaz:** La interfaz gráfica se construye como un conjunto de frames persistentes que se alternan en pantalla según la acción seleccionada, en lugar de abrir y cerrar ventanas nuevas. Esto evita reconstruir componentes innecesariamente. El frame activo se controla desde logica_interfaz.py mediante mostrar_frame().
9. **Un único archivo de partida guardada:** Para la funcionalidad opcional de guardar partida, se decidió manejar un único archivo que se sobrescribe en cada guardado. Esta decisión simplifica la implementación y evita la gestión de múltiples archivos.

Algoritmos usados

- **Búsqueda de rutas con backtracking y control de ciclos (ruta/3)** Para encontrar un camino entre dos módulos se implementó una búsqueda en profundidad mediante backtracking. El predicado ruta/3 llama a ruta_aux/4, que recibe un acumulador de

nodos ya visitados. En cada paso recursivo se extiende la ruta hacia un módulo adyacente que no esté en el acumulador, lo que garantiza que el algoritmo no entre en ciclos infinitos, aunque el grafo de módulos los contenga. El caso base se cumple cuando el nodo actual es igual al destino, momento en que se devuelve la lista acumulada como resultado.

- **Planificador de solución por simulación de estado (generar_plan/1)** El predicado como_gano/1 genera todos los planes posibles para ganar la partida. El núcleo es generar_plan/1, que opera sobre un estado simulado representado como el término estado(ModuloActual, ArtefactosUsados, SistemasReparados, ModulosVisitados) sin modificar el estado real del juego. El algoritmo identifica los sistemas pendientes de reparar y los tripulantes pendientes de rescatar, luego usa permutation/2 para explorar todos los órdenes posibles de atención. Para cada permutación genera la secuencia de pasos necesarios (moverse, tomar artefactos, usarlos, reparar, rescatar) propagando el estado simulado entre pasos. Finalmente, findall/3 recopila todos los planes válidos y list_to_set/2 elimina duplicados.
- **Verificación universal de objetivos con forall/2** La condición de victoria se verifica mediante cumple_objetivos_sistemas/0 y cumple_objetivos_tripulantes/0, que utilizan el predicado forall(Condicion, Accion). Este evalúa que para cada objetivo declarado (sistemas en objetivoS/2 y tripulantes en objetivoT/2) se cumpla su condición de estado correspondiente. El resultado es una verificación exhaustiva y declarativa sin necesidad de iterar manualmente sobre las listas.

Análisis de resultados

Objetivos alcanzados

Requerimientos

Requerimiento	Descripción	Implementado
RF-01	Definir los módulos de la estación mediante modulo/2	✓
RF-02	Definir enlaces bidireccionales entre módulos	✓
RF-03	Definir artefactos con gestión dinámica de estado	✓
RF-04	Definir sistemas reparables mediante sistema/4	✓

RF-05	Definir tripulantes mediante tripulante/4	✓
RF-06	Definir restricciones de acceso: necesita/2, necesitaEstado/3 y pasoPrevio/2	✓
RF-07	Mantener estado dinámico del jugador	✓
RF-08	Implementar tomar(Artefacto)	✓
RF-09	Implementar usar(Artefacto)	✓
RF-10	Implementar reparar(Sistema)	✓
RF-11	Implementar rescatar(Tripulante)	✓
RF-12	Implementar puedo_ir(Hacia)	✓
RF-13	Implementar mover(Modulo)	✓
RF-14	Implementar consultas: donde_esta/1, que_tengo/0, modulos_visitados/0	✓
RF-15	Implementar ruta(Inicio, Fin, Camino)	✓
RF-16	Implementar como_gano/0	✓
RF-17	Implementar verifica_gane/0	✓
RF-18	Implementar describir_modulo/0	✓
RF-19	Implementar interfaz gráfica conectada con Prolog	✓
RF-20	Guardar y reproducir una partida en disco (Opcional)	✓

Resumen de Implementación

Total de requerimientos	20
Requerimientos obligatorios implementados	19 / 19
Requerimientos opcionales implementados	1 / 1 (RF-20: Guardar y reproducir partida)
Estado general	✓ Todos fueron implementados

Objetivos no alcanzados y razones

Todos los objetivos obligatorios del proyecto fueron completados satisfactoriamente. No quedaron funcionalidades requeridas por el enunciado pendientes de implementación.

Análisis de predicados

Explicación de los tres predicados más complejos

Generar_plan/1:

El predicado generar_plan/1 es el núcleo del planificador de victoria. Su complejidad radica en que no opera sobre el estado real del juego sino sobre un estado simulado representado como el término estado(ModuloActual, ArtefactosUsados, SistemasReparados, ModulosVisitados), lo que le permite explorar secuencias de acciones sin alterar la partida en curso.

El algoritmo comienza consultando el estado actual del jugador para construir el estado inicial de simulación. Luego identifica mediante findall/3 los sistemas pendientes de reparar y los tripulantes pendientes de rescatar según los objetivos declarados en objetivoS/2 y objetivoT/2. Sobre estas listas aplica permutation/2 dos veces, generando todas las combinaciones posibles de orden de atención. Para cada permutación invoca pasos_para_sistemas_ord/4 y pasos_para_tripulantes_ord/4, que propagan el estado simulado de paso en paso generando la secuencia completa de acciones necesarias. El resultado es un plan concreto que incluye movimientos, toma de artefactos, usos, reparaciones y rescates en el orden correcto. Finalmente, como_gano/1 usa findall/3 para recopilar todos los planes válidos generados por backtracking y list_to_set/2 para eliminar duplicados.

Ruta_aux/4:

El predicado ruta_aux/4 implementa una búsqueda en profundidad con backtracking para encontrar un camino simple entre dos módulos del grafo de la estación. Su diseño maneja correctamente grafos con ciclos sin caer en recursión infinita.

El predicado recibe el módulo actual, el módulo destino, la lista de módulos ya visitados en el recorrido actual y unifica el cuarto argumento con la ruta completa cuando la encuentra. El caso base se dispara cuando el módulo actual es igual al destino, retornando directamente la lista acumulada. En el caso recursivo, el predicado consulta esta_conectado/2 para obtener un módulo adyacente, verifica con \+ member/2 que ese módulo no haya sido visitado ya en el recorrido actual, lo agrega al acumulador y recursa. Si Prolog no logra llegar al destino por ese camino, el backtracking deshace la elección y prueba con otro módulo adyacente. Este mecanismo garantiza que la búsqueda explore todas las rutas posibles sin repetir nodos, devolviendo la primera que conecte origen con destino.

Pasos_recorrer_nodos/4:

El predicado `pasos_recorrer_nodos/4` es el componente más complejo del planificador, encargado de generar la secuencia de pasos para recorrer una lista de módulos respetando todas las restricciones de acceso del juego.

Para cada módulo de la lista verifica primero si existe algún artefacto requerido para entrar que aún no haya sido usado en el estado simulado. Si existe, invoca `pasos_obtener_artefacto/4` para generar los pasos de conseguirlo y usarlo antes de continuar, actualizando el estado simulado con el artefacto ya disponible. A continuación, verifica mediante `simulacion_puede_entrar/4` que con el estado actualizado sea posible ingresar al módulo, considerando las restricciones de `necesita/2`, `necesitaEstado/3` y `pasoPrevio/2`. Si el módulo actual del estado simulado no es el destino, calcula una ruta intermedia con `ruta/3` y genera los pasos de cada salto con `pasos_salto_directos/4`. Finalmente actualiza la lista de módulos visitados en el estado simulado y recursa sobre el resto de la lista. Este encadenamiento de resolución de prerequisites, cálculo de rutas intermedias y propagación de estado simulado es lo que hace al predicado el más complejo del sistema.

Bitácora

Registro de commits en el repositorio: <https://github.com/MaikelFn/Operacion-Atlas>